기계화 명세 기반
이더리움 합의 클라이언트 비교 테스팅

SpecTrum: Specification-Guided Differential Fuzzing for Ethereum Consensus Clients

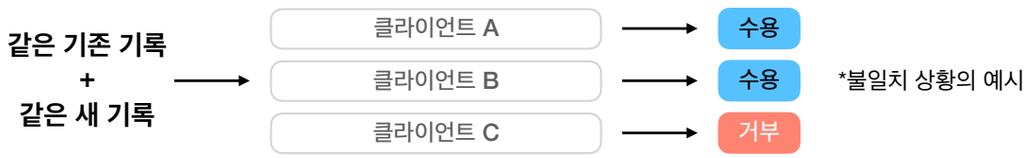
정석훈, 단경민, 류석영, 황성재 (ASE'26)

안녕하세요. 류석영 교수님 연구실 석사과정 정석훈입니다. 지금부터 기계화 명세 기반 이더리움 합의 클라이언트 비교 테스팅 주제로 발표드리겠습니다. 해당 연구는 저희 지도교수님, 그리고 성균관대 황성재 교수님, 단경민님과 함께했으며, 올해 ASE에서 발표예정입니다. (20s)

이더리움에서는 클라이언트 불일치가 네트워크를 분열

합의:
어떤 기록을 공동의 기록으로 인정할지 결정

클라이언트 다양성:
한 구현체 버그의 네트워크 영향을 줄이기 위해
같은 합의 규칙을 구현한 여러 클라이언트를 함께 사용



*불일치 상황의 예시

유효하다고 인정하는 기록이 달라짐 → 네트워크 분열 가능

이더리움에서는 프로그램 사이의 동작 차이가 네트워크의 분열로 이어질 수 있습니다.

이더리움은 여러 컴퓨터가 거래 기록을 공동으로 관리하는 분산시스템입니다. 이 때 여러 컴퓨터가 어떤 기록을 공동의 기록으로 인정할지 결정하는 과정을 합의라고 합니다.
각 컴퓨터는 공통의 합의 규칙에 따라 새로운 기록을 받아들일지 판단합니다. 이 합의 규칙을 실행하는 프로그램을 합의 클라이언트라고 하고, 이번 발표에서는 줄여서 클라이언트라고 부르겠습니다.

이더리움에서는 여러 팀이 서로 다른 언어로 개발한 클라이언트를 한 네트워크 위에서 사용합니다. 하나의 클라이언트에 생긴 버그가 네트워크 전체에 영향을 줄 위험을 줄이기 위해서입니다.

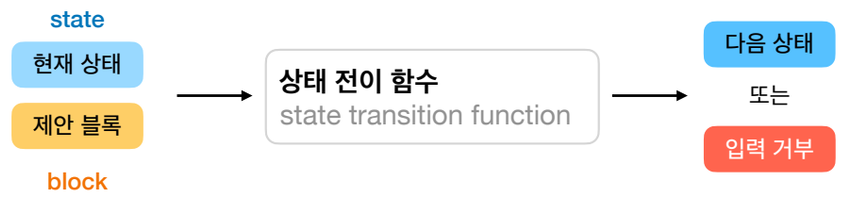
그런데 같은 기존 기록을 가진 컴퓨터들이 동일한 새 기록을 처리할 때, 그림처럼 일부 클라이언트는 받아들이고 다른 클라이언트는 거부하는, 클라이언트간 동작의 불일치가 일어날 수 있습니다.

그러면 한쪽이 유효하다고 인정한 기록을 다른쪽은 인정하지 않아, 서로 다른 기록을 따르는 네트워크 분열이 일어납니다.

저희는 같은 규칙을 구현한 클라이언트들이 같은 입력을 다르게 처리하는 불일치를 찾아내는 연구를 했습니다.

검사 대상은 블록을 처리하는 상태 전이 함수

블록: 새 기록을 담아 제안하는 단위
상태: 블록 처리에 필요한 정보 — 검증자 목록, 잔액 등



이 연구에서는 클라이언트가 새 블록을 처리할 때 실행하는 상태 전이 함수를 검사합니다.

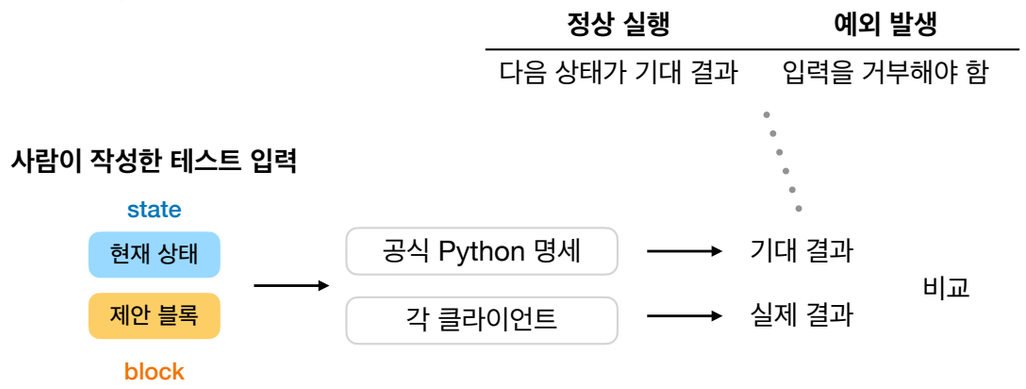
앞서 말한 새 기록은 블록이라는 단위로 추가됩니다.

클라이언트는 블록 처리에 필요한 정보를 상태로 관리합니다. 여기에는 합의에 참여하는 검증자의 목록과 잔액 등이 들어 있습니다.

상태 전이 함수는 현재 상태와 제안된 블록을 입력으로 받습니다. 정상적으로 처리하면 블록을 반영한 다음 상태를 계산하고, 유효하지 않은 입력은 거부합니다.
—> 따라서 같은 상태와 블록에 대해서는 입력의 거부 여부가 같아야 하고, 정상적으로 처리했다면 다음 상태도 같아야 합니다.

공식 명세가 기대 결과 결정, 공식 테스트가 구현 검사

consensus-spec



클라이언트가 따라야 할 기대 결과는 실행 가능한 공식 명세로 계산합니다.

앞서 본 상태 전이 함수의 동작은 consensus-spec이라는 공식 명세에 정의되어 있습니다. 마크다운 문서 안에 Python 코드가 들어 있으며, 이 코드를 직접 실행할 수 있습니다.

현재 상태와 제안된 블록을 넣으면, 정상 실행에서는 다음 상태를 얻습니다. 예외가 발생하면 해당 입력을 거부해야 한다는 뜻입니다. 따라서 명세를 실행하면 클라이언트가 내야 할 결과를 알 수 있습니다.

공식 테스트도 이 명세를 이용합니다. 사람이 테스트 입력을 작성하고, 명세를 실행해 기대 결과를 계산합니다. 그런 다음 클라이언트에도 같은 입력을 넣어 결과를 비교합니다.

—> 공식 테스트는 이렇게 준비된 입력에 대해, 클라이언트의 결과가 명세와 같은지 확인합니다.

공식 명세가 기대 결과 결정, 공식 테스트가 구현 검사

consensus-spec

Beacon chain state transition function

The post-state corresponding to a pre-state `state` and a signed block `signed_block` is defined as `state_transition(state, signed_block)`. State transitions that trigger an unhandled exception (e.g. a failed `assert` or an out-of-range list access) are considered invalid. State transitions that cause a `uint64` overflow or underflow are also considered invalid.

```
def state_transition(
    state: BeaconState, signed_block: SignedBeaconBlock, validate_result: b
) -> None:
    block = signed_block.message
    # Process slots (including those with no blocks) since block
    process_slots(state, block.slot)
    # Verify signature
    if validate_result:
        assert verify_block_signature(state, signed_block)
    # Process block
    process_block(state, block)
    # Verify state root
    if validate_result:
        assert block.state_root == hash_tree_root(state)
```

사람이 작성한

st

현재

제안

bl

비교

클라이언트가 따라야 할 기대 결과는 실행 가능한 공식 명세로 계산합니다.

앞서 본 상태 전이 함수의 동작은 consensus-spec이라는 공식 명세에 정의되어 있습니다. 마크다운 문서 안에 Python 코드가 들어 있으며, 이 코드를 직접 실행할 수 있습니다.

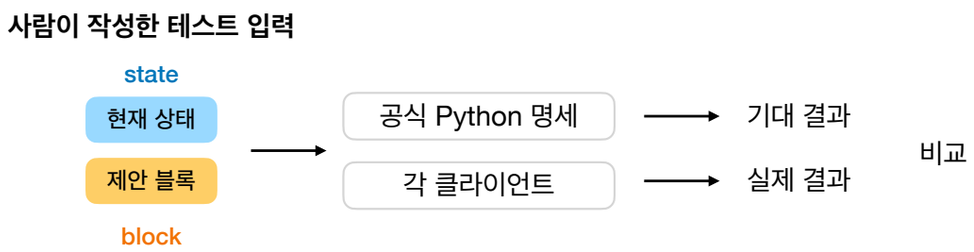
현재 상태와 제안된 블록을 넣으면, 정상 실행에서는 다음 상태를 얻습니다. 예외가 발생하면 해당 입력을 거부해야 한다는 뜻입니다. 따라서 명세를 실행하면 클라이언트가 내야 할 결과를 알 수 있습니다.

—> 공식 테스트도 이 명세를 이용합니다. 사람이 테스트 입력을 작성하고, 명세를 실행해 기대 결과를 계산합니다. 그런 다음 클라이언트에도 같은 입력을 넣어 결과를 비교합니다.

공식 테스트는 이렇게 준비된 입력에 대해, 클라이언트의 결과가 명세와 같은지 확인합니다.

공식 명세가 기대 결과 결정, 공식 테스트가 구현 검사

consensus-spec



클라이언트가 따라야 할 기대 결과는 실행 가능한 공식 명세로 계산합니다.

앞서 본 상태 전이 함수의 동작은 consensus-spec이라는 공식 명세에 정의되어 있습니다. 마크다운 문서 안에 Python 코드가 들어 있으며, 이 코드를 직접 실행할 수 있습니다.

현재 상태와 제안된 블록을 넣으면, 정상 실행에서는 다음 상태를 얻습니다. 예외가 발생하면 해당 입력을 거부해야 한다는 뜻입니다. 따라서 명세를 실행하면 클라이언트가 내야 할 결과를 알 수 있습니다.

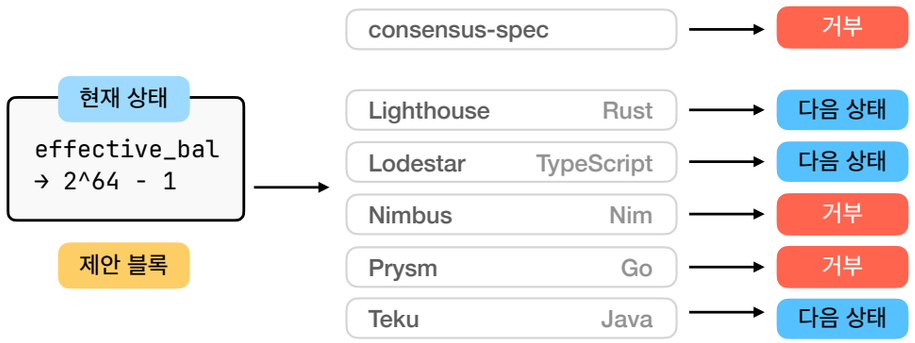
—> 공식 테스트도 이 명세를 이용합니다. 사람이 테스트 입력을 작성하고, 명세를 실행해 기대 결과를 계산합니다. 그런 다음 클라이언트에도 같은 입력을 넣어 결과를 비교합니다.

공식 테스트는 이렇게 준비된 입력에 대해, 클라이언트의 결과가 명세와 같은지 확인합니다.

공식 테스트에 없는 입력에서 불일치 발생

입력 생성: 공식 테스트에서 상태의 잔액 필드 하나를 `2^64 - 1` 로 변경

관찰 결과: 명세 및 다섯 클라이언트에 실행했을 때 불일치 발생



그러나 공식 테스트가 검사하지 않은 입력에서 명세와 클라이언트가 입력의 거부 여부를 다르게 판단했습니다.

저희는 기존 테스트 입력에서 잔액 필드 하나를 uint64가 표현할 수 있는 최댓값으로 바꿨습니다.
이 입력을 원본 명세와, 이번 연구 대상인 다섯 클라이언트에서 실행했습니다.
명세는 이 입력을 거부했지만 다섯 클라이언트 중 세 개는 받아들였습니다.

—> 명세에는 이 입력을 거부해야 한다고 동작이 정의되어 있었지만, 그 상황을 검사하는 입력은 공식 테스트에 없었던 것입니다.

테스트가 빠진 이유는 명세에 숨은 예외 조건 때문

테스트: 작성자가 검사할 경우를 정해 **입력 작성**

↓

명세: 명세를 실행해 **기대 결과 계산**

consensus-spec

```
if prev_bal * 3 >= total_bal * 2:
```

uint64 연산이 범위 초과

↓

overflow 예외 발생 ⟶ 입력 거부

숨은 예외 조건: 연산·타입의 동작에 포함되지만, 별도의 검사로 적혀 있지 않은 예외 조건

이 조건을 겨냥하려면 작성자가 연산과 타입의 동작에서 조건을 찾아야 함

공식 테스트가 불일치를 놓친 이유는 명세에 예외 조건이 숨어있었기 때문입니다.
공식 명세는 입력이 주어졌을 때의 기대 결과를 계산합니다. 하지만 어떤 경우를 검사할지 정하고 그에 맞는 입력을 만드는 일은 테스트를 작성하는 사람이 수행합니다.
앞서 본 사례에서는 uint64 연산의 결과가 표현 범위를 벗어나면 overflow 예외가 발생하고, 입력을 거부해야 합니다.
상태 전이 코드에는 잔액 계산이 적혀 있지만, overflow가 발생하는 조건을 확인하는 검사는 따로 적혀 있지 않습니다. 이처럼 연산이나 타입의 동작에 포함되어 있지만, 상태 전이 코드에 별도의 검사로 적혀 있지 않은 예외 조건을 ‘숨은 조건’이라고 부르겠습니다.
이런 조건을 겨냥한 테스트를 작성하려면, 작성자가 연산과 타입의 동작에서 해당 조건을 직접 찾아야 합니다. 그 조건을 놓치면 해당 상황을 검사하는 입력도 빠질 수 있습니다.
—> 이렇게 명세가 요구하는 동작 중 일부가 기존 테스트에서 검사되지 않은 채 남을 수 있습니다.

테스트가 안 된 조건을 목표로 입력 생성

필요한 정보	얻는 방법
목표 조건	**명세를 풀어써 조건을 명시**하고, 기존 테스트의 **조건 커버리지 측정**
바꿀 기존 입력	목표 조건에 도달한 **기존 테스트 선택**
바꿀 필드	해당 실행에서 값의 **유래 추적**
넣을 값	목표 조건을 이용해 **변경 후보** 생성

따라서 저희는 기존 테스트가 검사하지 않은 조건을 겨냥해, 불일치를 드러낼 입력을 만듭니다.

입력을 새로 만드는 것은 어려우므로 기존의 공식 테스트를 변이시킬 건데요, 이때 필요한 정보는 네 가지입니다. 어떤 조건을 겨냥할지, 어떤 기존 입력을 사용할지, 그 입력의 어느 필드를 바꿀지, 그리고 어떤 값을 넣을지입니다.

- 먼저 명세를 풀어써 실행 결과를 결정하는 경로 조건을 명시하고, 앞서 본 숨은 조건도 명시적인 검사로 표현합니다. 기존 테스트에서 각 검사가 어떻게 평가됐는지 측정해 목표 조건과 기존 입력을 고릅니다.
- 다음으로 선택한 입력의 실행을 따라, 목표 조건에 사용된 값이 어느 입력 필드에서 왔는지 추적합니다. 이렇게 바꿀 필드를 찾은 뒤에는 목표 조건의 비교 기준을 이용해 넣을 값의 후보를 정합니다.
- 마지막으로 생성한 입력을 원본 파이썬 명세와 다섯 클라이언트에서 실행해, 입력의 거부 여부와 최종 상태가 같은지 비교합니다.

—> 먼저 목표 조건을 고르기 위해, 기존 테스트를 측정하고 구분하는 기준부터 정하겠습니다.

기존 테스트가 어떤 실행 결과를 검사했는지 구분

실행 결과: 함수의 반환값과 상태 갱신을 포함

consensus-spec (python)

```python
if P:
  state.curr_jcp = prev_epoch
if Q:
  state.curr_jcp = curr_epoch
```

P	Q	실행 결과	테스트
참	참	S[curr_jcp ← curr_epoch]	실행
거짓	거짓	S	실행
참	거짓	S[curr_jcp ← prev_epoch]	미실행
거짓	참	S[curr_jcp ← curr_epoch]	미실행

분기 커버리지

경로 커버리지

목표 조건을 고르기 위해, 먼저 기존 테스트가 함수의 어떤 실행 결과를 검사했는지 살펴보겠습니다.

- 이 때 실행 결과는 반환값과 상태 갱신을 포함합니다.
- 이 코드는 공식 명세에서 상태의 한 항목을 갱신하는 부분입니다. [두 비교식은 P와 Q로 표시했습니다. 첫 번쨰 if문은 curr_jcp에 prev_epoch을 저장하고, 두 번째 if문은 같은 항목에 curr_epoch을 저장합니다.]
- 두 조건이 모두 참인 입력을 실행하면 curr_jcp가 curr_epoch으로 바뀝니다. 또한 두 조건이 모두 거짓이면 기존 상태를 유지합니다. 각 분기의 실행 여부를 확인하는 분기 커버리지 관점에서는, 이렇게 두 입력만 실행해도 모든 비교문의 참/거짓 분기를 다 실행해봤습니다. 하지만 실제로는 curr_jcp가 prev_epoch이 되는 실행 결과는 한 번도 일어나지 않았습니다.
- 이런 빠진 실행 결과까지 검사하려면 각 입력이 거친 경로 조건을 구분하는 경로 커버리지가 필요합니다.

—> 저희는 경로 커버리지에서 착안해, 실행 결과마다 어떤 경로 조건이 성립해야 하는지를 명세에 직접 적습니다.

경로 조건과 실행 결과를 함께 명시

경로 조건: 특정 실행 결과를 일으키기 위해 입력이 만족해야 하는 조건

모든 정상 실행은 정확히 하나의 행에 해당
(경로 조건끼리 서로소, 완전)

경로 조건	실행 결과
Q	S[curr_jcp ← curr_epoch]
P ∧ ¬Q	S[curr_jcp ← prev_epoch]
¬P ∧ ¬Q	S

consensus-spec (python)

```python
if P:
  state.curr_jcp = prev_epoch
if Q:
  state.curr_jcp = curr_epoch
```

Consensus-SpecTec

```
state ~> state[.CURR_JCP = prev_epoch]
-- if P
-- if ~Q
```


경로의 조건과 실행 결과를 명세에 직접 적고, 그 조건을 커버리지 측정과 입력 변이의 기준으로 삼습니다.

- 입력이 특정 실행 결과를 일으키기 위해 만족해야 하는 조건을 경로 조건이라고 합니다.
- 예를 들어 두 번째 행을 보면, prev_epoch을 저장하는 경로를 실행하려면 P가 참이고 Q가 거짓이어야 합니다.
- 다른 경로도 같은 방식으로 표현합니다. 두 조건이 모두 거짓이면 실행 결과는 기존 상태입니다. 또한 Q가 참이면 P의 값과 관계 없이 실행 결과가 curr_epoch으로 동일하므로, 이 두 경로는 하나의 행으로 묶습니다. 이렇게 결과가 같으면 묶는다는 점에서 기존 경로 커버리지와 다릅니다.
- 정상 실행에서는 각 입력이 정확히 하나의 행에 해당하도록 구성했습니다. 두 행에 동시에 해당하는 입력도, 어느 행에 해당되지 않는 입력도 없습니다. 즉 서로소이면서 완전합니다.
- 이렇게 하면 정상 실행에서 어떤 조건이 어떤 결과로 이어지는지 명세에 직접 드러납니다.

—> 다음으로는 문제에서 본 숨은 조건도 명시적인 검사로 표현하겠습니다.

입력을 거부하는 예외 상황도 조건으로 표현

consensus-spec

```
1 if prev_bal * 3 >= total_bal * 2:
```

Consensus-SpecTec

```
1 -- if total_bal <= UINT64_MAX / 2
2 -- if prev_bal * 3 >= total_bal * 2
```

곱셈이 overflow를 일으켜도 입력 거부

참	곱셈의 결과가 uint64 범위 안에 포함
거짓	곱셈시 overflow가 발생하므로 입력 거부

예외 방지 조건

입력을 거부해야 하는 예외 상황도 조건의 참과 거짓으로 구분합니다.

- 이 코드는 앞서 P로 표현한 비교식입니다. 두 값의 크기를 비교하려면 먼저 곱셈을 계산해야 합니다. 여기서 total_bal은 명세가 계산한 잔액 합계이며, uint64로 표현됩니다.
- 이 값의 두 배가 uint64 범위를 벗어나면 overflow 예외가 발생하며, 명세에 따르면 입력을 거부해야 합니다. 범위를 벗어나지 않으려면 원래 값이 uint64 최댓값의 절반 이하여야 하므로, 저희는 이 조건을 명세에 직접 적습니다.
- 이 조건이 참이면 해당 곱셈은 범위 안에서 계산 되고, 거짓이면 overflow가 발생하므로 입력을 거부합니다.

- 풀어쓴 명세는 P4-SpecTec으로 작성했습니다. P4-SpecTec으로 작성한 명세는 바로 실행가능해서, 상태 전이 테스트와 단위 테스트를 합친 공식 테스트 910개에서 원본 Python 명세와 비교했을 때 두 명세 모두 예외를 일으키거나 정상 출력이 동일했습니다.

—> 이제 정상 실행을 구분하는 조건과 에외 발생 여부를 구분하는 조건이 모두 명세에 드러났습니다.

풀어쓴 명세를 실행해 공식 테스트를 통과

consensus-spec / weigh_justification_and_finalization (simpl.)

```
def weigh_just_and_fin(
      state, total_bal,
      prev_bal, curr_bal
    ):
if prev_bal * 3 >= total_bal * 2:
  state.curr_jcp = prev_epoch
if curr_bal * 3 >= total_bal * 2:
  state.curr_jcp = curr_epoch
```

Consensus-SpecTec: WeighJustificationAndFinalization (simpl.)

```
rule WeighJustAndFin/justify_prev:
  state, total_bal, prev_bal, curr_bal
    ~> state[.CURR_JCP = prev_epoch]
-- if total_bal <= UINT64_MAX / 2
-- if prev_bal <= UINT64_MAX / 3
-- if curr_bal <= UINT64_MAX / 3
-- if prev_bal * 3 >= total_bal * 2
-- if ~(curr_bal * 3 >= total_bal * 2)
```

*나머지 두 규칙은 생략

명세 언어: P4-SpecTec으로 작성하여 실행 가능

테스트 통과: 공식 테스트 910개에서 원본 Python 명세와 동작 일치 (출력값 일치 또는 둘 다 예외 발생)

풀어쓴 명세를 실행해서, 공식 테스트를 통과함을 확인했습니다.

오른쪽 코드는 앞서 설명한 경로 조건과 숨어있던 예외 방지 조건을 함께 적은 예시 명세입니다. 왼쪽에 있는 공식명세 함수의 세 규칙 중 하나이며, P4-SpecTec 명세언어로 작성했습니다.

P4-SpecTec으로 작성한 명세는 실행가능하기 때문에 입력에 대한 결과를 계산할 수 있습니다. 저희는 명세를 풀어쓰면서 실행 결과가 달라지지 않았는지 확인하기 위해, 같은 공식 테스트 입력을 원본 Python 명세와 풀어쓴 명세에 각각 넣었습니다.

상태 전이 테스트와 단위 테스트를 합친 910개 모두에서, 정상적으로 실행을 마쳤을 때는 출력이 같았고, 예외가 발생한 경우에는 양쪽 모두 예외가 발생했습니다.

이렇게 작성하고 확인한 명세를 이후 커버리지 측정과 테스트 생성에 사용합니다.

조건 커버리지로 목표 조건과 기존 입력 선택

조건 커버리지: 명세의 각 조건이 참/거짓으로 실행됐는지 측정
측정 대상 입력: 공식 상태 전이 테스트 581개

Consensus-SpecTec: WeighJustificationAndFinalization (simpl.)

```
1 rule WeighJustAndFin/justify_prev:
2   state, total_bal, prev_bal, curr_bal
      ~> state[.CURR_JCP = prev_epoch]
3 -- if total_bal <= UINT64_MAX / 2
4 -- if prev_bal <= UINT64_MAX / 3
5 -- if curr_bal <= UINT64_MAX / 3
6 -- if prev_bal * 3 >= total_bal * 2
7 -- if ~(curr_bal * 3 >= total_bal * 2)
```

참	거짓
21회	0회

목표 조건: 참 > 0회, 거짓 = 0회

- 거짓이 될 수 없는 조건은 수동으로 제외

바꿀 기존 입력: 목표 조건에 도달한 입력

조건 커버리지를 측정해 아직 거짓으로 평가되지 않은 조건을 목표로 선택합니다.

- 조건 커버리지는 명세에 각 조건이 참과 거짓으로 평가됐는지 확인하는 지표입니다. 한 조건에서 두 값이 모두 평가되었으면 그 조건을 완전히 검사한 것으로 봅니다.
- 공식 상태 전이 테스트 581개를 풀어쓴 명세에서 실행하고, 각 조건이 참과 거짓으로 평가된 횟수를 기록합니다. 같은 조건이 여러 경우에 등장하면 각각 따로 기록합니다.
- 앞서 본 overflow 방지 조건은 참으로 21번 평가됐지만, 거짓으로는 한 번도 평가되지 않았습니다. 이 조건을 검사한 실행은 있었지만, overflow가 발생하는 쪽은 검사하지 않은 것입니다.
- 다만 해당 위치에 도달하는 실행에서는 항상 참일 수밖에 없는 조건도 있습니다. (infeasible test requirement) 이런 조건은 수동으로 확인해 제외합니다.
- 목표 조건을 정한 뒤에는 그 조건에 도달했던 기존 테스트 입력을 고릅니다. 이 입력을 새 테스트를 만드는 출발점, 즉 시드로 사용합니다.

—> 여기까지 얻은 정보는 목표 조건과, 그 조건에 도달했던 입력입니다. 다음으로는 이 입력의 어느 필드가 목표 조건에 영향을 주었는지 찾습니다.

값의 유래를 추적해 목표 조건 관련 항목 수집

유래 정보: 값을 도출하는 데에 사용된 입력 필드 경로의 집합

$S[p]$: 상태 S에서 경로 p에 저장된 값

```
p₀ = VALIDATORS[0].EFFECTIVE_BAL
p₃ = VALIDATORS[3].EFFECTIVE_BAL
```

$\Pi(S[p_0]) = \{p_0\}$

$\Pi(S[p_3]) = \{p_3\}$

$S[p_0] + S[p_3] + \ldots$

↓ 합산

`total_bal`

↓

목표 조건:

```
-- if total_bal <= UINT64_MAX / 2
```

유래 정보 전달

$\Pi(e_1 + e_2) = \Pi(e_1) \cup \Pi(e_2)$

$\Pi(\texttt{total_bal}) = \{p_0, p_3, \ldots\}$

목표 조건에 사용된 값의 유래를 추적해, 바꿀 입력 필드의 경로를 찾습니다.
앞서 본 total_bal은 입력 상태에 있는 여러 잔액을 더해 계산한 값입니다. 이 합계를 바꾸려면, 합계에 기여한 잔액이 입력의 어디에 저장되어 있는지 알아야 합니다.

여기서 p_0는 validators 리스트에서 인덱스가 0인 원소의 effective_balance 필드를 가리킵니다. 이렇게 필드 이름과 리스트 인덱스로 입력의 위치를 나타낸 것을 필드 경로라고 하겠습니다. $S[p_0]$는 입력 상태의 그 위치에 저장된 값입니다.
명세 실행 도중에 값을 생성할 때마다, 그 값이 나온 필드 경로를 함께 기록합니다. 가령 두 값을 더하면, 합계에는 두 값의 유래를 합쳐 기록합니다. 여기서는 두 경로 p_0와 p_3가 함께 남습니다.
이 정보를 계산 과정에 따라 전달하면, 목표 조건에서 total_bal을 검사할 때 합계에 기여한 입력 필드들의 경로를 얻을 수 있습니다. 이것이 유래 추적입니다.
이렇게 얻은 경로가 입력의 어디를 바꿔야 할지 알려줍니다.
—> 여기서는 p_0를 바꿀 필드로 정하고, 이제 p_0가 가리키는 필드에 넣을 값을 정하겠습니다.

목표 조건으로 변경할 값 선택 및 변이, 실행 결과 비교

바꿀 필드: p_0

목표 조건: `-- if total_bal <= θ` $M = \texttt{UINT64_MAX}$ $\theta = \left\lfloor \frac{M}{2} \right\rfloor$

만족시킬 조건: `total_bal > θ`

넣을 값 범위: $[\theta + 1, M]$

값 선택: 범위의 양 끝, 내부, 바로 바깥 $V = \left\{ \theta, \theta+1, \theta+1+\left\lfloor \frac{\theta}{3} \right\rfloor, \theta+1+\left\lfloor \frac{2\theta}{3} \right\rfloor, M, \cancel{M+1} \right\}$

$S' = S[p_0 \leftarrow v] \quad (v \in V)$ $S[p_0 \leftarrow v]$: S에서 경로 p_0의 값만 v로 바꾼 새 상태

실행 결과 비교: 각 새 상태를 원래 제안 블럭과 함께 **공식 명세 + 다섯 클라이언트**에서 실행
비정상 종료 / 입력 거부 / 다음 상태 비교

목표 조건에서 후보값을 고르고, 선택한 경로의 필드만 바꿔 새 입력을 만듭니다.
앞서 유래 추적으로 얻은 경로 중 p_0를 선택했습니다. 이제 이 위치에 저장된 잔액을 어떤 값으로 바꿀지 정합니다.
목표는 total_bal이 uint64 최댓값의 절반 이하여야 한다는 조건을 거짓으로 만드는 것입니다. 이 비교 기준을 세타라고 하겠습니다.
저희는 이 기준을 p_0가 가리키는 잔액 필드에 적용해, 그 값 자체를 세타보다 크게 바꾸는 후보를 만듭니다. 이 필드도 uint64이므로 후보 구간은 세타보다 큰 값부터 uint64 최댓값까지입니다.
값을 선택할 때는 각 구간의 양 끝 값과, 내부에서 두 값을 고르고, 구간 바로 바깥도 함께 선택합니다. 최댓값 바로 바깥은 범위 초과이므로 제외하고, 남은 다섯 값이 집합 V입니다.
각 후보값마다 원래 상태에서 p_0가 가리키는 항목만 바꿉니다. 이 표기는 그 항목의 값을 v로 바꾼 새 상태를 뜻합니다. p_3를 비롯한 다른 위치의 값은 유지됩니다.
이렇게 만든 상태를 원래 제안 블록과 함께 실행합니다. 입력을 바꾸면 목표 조건에 도달하기 전에 다른 연산이나 조건에서 실패할 수도 있으므로, 실제 실행 결과를 확인합니다.

원본 Python 명세와 다섯 클라이언트에서 비정상 종료, 입력의 거부 여부, 정상 실행의 다음 상태를 비교합니다.
—> 유래 추적으로 입력에서 바꿀 위치를 찾고, 목표 조건으로 그 위치에 넣을 값을 정해, 불일치를 검사할 새 입력을 만드는 것입니다.

① 생성한 테스트로 불일치 27개를 발견

실험 범위: Capella · 다섯 클라이언트 · 생성한 테스트 21,444개

Category	Divergences	
입력 조건 위반	●●●●●●●●●●●	11
정수 범위 초과 처리	●●●●●●●	7
정수 디코딩 오류	●●●●	4
함수 호출 조건 위반	●●●	3
프로토콜 버전 선택 오류	●	1
계산 결과 불일치	●	1
		27

같은 구현 결함과 클라이언트별 반응을 기준으로 분류

23개는 블럭 만들어 보낼 때만
4개는 받은 블럭 검사할 때도

생성한 2만여개(21,444) 테스트로 다섯 클라이언트를 비교해, 27개의 불일치 사례를 발견했습니다.

같은 구현 결함에서 비롯되고, 명세와 클라이언트별 반응도 같은 불일치 입력들을 하나의 사례로 묶었습니다.
그림에서 동그라미 하나가 하나의 사례이고, 원인의 종류에 따라 여섯 가지로 분류했습니다. 앞서 본 잔액 예시처럼 상태가 지켜야 할 조건을 위반한 입력을 서로 다르게 처리한 경우도 있었고, 산술 연산의 overflow나 underflow를 다르게 처리하거나, 정수를 읽어 들이는 과정에서 값을 다르게 해석하는 경우도 발견했습니다.
이 불일치는 어떤 검사를 수행하느냐에 따라서도 달라졌습니다. 클라이언트는 새 블록을 만들어 보낼 때도, 다른 곳에서 받은 블록을 검사할 때도 상태 전이 함수를 실행합니다. 블록을 만들 때 사용하는 설정에서는 일부 검사를 생략하고, 받은 블록을 검사할 때 사용하는 설정에서는 그 검사까지 수행합니다.
23개는 블록을 만들 때 사용하는 설정에서만 불일치가 나타났습니다. 나머지 4개는 받은 블록을 검사할 때 사용하는 설정에서도 불일치가 나타났습니다.
이렇게 기존 테스트 입력을 변이해, 공식 테스트에서 드러나지 않았던 클라이언트 간 동작 차이를 찾았습니다.
—> 다음으로, 이 테스트들이 어떤 조건을 추가로 검사했는지 살펴보겠습니다.

② 조건 126개를 추가로 거짓으로 평가

	(46.3%)	(**91.1%**)
조건 커버리지 중 거짓 비율	130/281	→ 256/281 (+126)
파이썬 명세 분기 커버리지	152/358	→ 154/358 (+2)

추가로 검사한 조건 126개 중 **65개**는 명세를 풀어쓰면서 추가한 예외 방지 조건

불일치 27개 중 **22개**는 새로 추가한 조건을 겨냥한 테스트에서 발견

생성한 테스트는 기존 테스트에서 거짓으로 평가된 적 없던 조건 126개를 추가로 거짓으로 만들었습니다.
앞서 조건 커버리지를 측정할 때, 각 조건이 참과 거짓으로 평가됐는지를 기록했습니다. 여기서는 그중 거짓으로 평가된 적 있는 조건의 비율을 비교했습니다.
항상 참일 수밖에 없는 조건을 제외하면, 기존 공식 테스트에서는 약 46%의 조건이 거짓으로 평가됐습니다. 생성한 테스트를 추가하면 이 비율이 약 91%로 증가합니다.
같은 테스트를 원본 Python 명세에서 실행했을 때, 새롭게 실행된 분기는 두 개뿐이었습니다. 추가로 검사한 조건은 많았지만, 그 변화가 분기 커버리지에는 거의 나타나지 않았습니다.
추가로 검사한 126개 중 65개는 명세를 풀어쓰면서 직접 적은 예외 방지 조건입니다. 앞서 곱셈의 overflow를 막는 조건을 적었던 것처럼, 연산에 숨어 있던 조건을 명세에 드러낸 것입니다. 이 조건이 거짓으로 평가됐다는 것은 해당 예외 상황을 검사했다는 뜻입니다.
또한 앞서 보여드린 불일치 27개 중 22개는 명세에 추가한 조건을 겨냥한 테스트에서 발견했습니다. 잔액을 바꾼 예시도 여기에 해당합니다.
명세에 조건을 직접 적고 그 평가 결과를 기록하면, 분기 커버리지의 변화에 나타나지 않는 검사까지 확인할 수 있습니다. 그리고 그 조건을 겨냥한 테스트로 실제 불일치도 찾을 수 있었습니다.

③ 같은 테스트 수로 다른 방법보다 많은 불일치 발견

각 방법에서 테스트 21,444개 생성

방법	필드와 값 선택 방법	발견한 불일치
Random	필드, 값 모두 무작위	12
Extreme	필드는 무작위, 값은 경계값	16
SpecTrum (이 연구)	명세의 조건과 값의 유래를 활용	27

Random, Extreme 각각 SpecTrum이 놓친 사례도 5개씩 발견

명세의 조건과 값의 유래를 활용한 저희 방법이, 같은 수의 테스트로 두 가지 다른 방법보다 더 많은 불일치를 찾았습니다.

앞서 본 예시에서는 잔액을 큰 값으로 바꿨습니다. 이런 입력은 명세를 분석하지 않아도 큰 값이나 경계값을 선택하는 방법으로 만들 수도 있습니다. 그래서 명세의 조건과 값의 유래를 활용하는 것이 얼마나 도움이 되는지 비교했습니다.

비교한 두 방법도 기존 테스트 입력에서 필드 하나를 바꿉니다. Random은 필드와 값을 무작위로 고릅니다. Extreme은 필드를 무작위로 고르되, 그 타입의 최댓값이나 최솟값 같은 경계값을 넣습니다.
세 방법 모두 21,444개 테스트를 생성했습니다. Random은 불일치 12개, Extreme은 16개, 저희 도구인 SpecTrum은 27개를 발견했습니다.
다만 발견한 사례가 모두 겹치지는 않았습니다. Random과 Extreme도 각각 SpecTrum이 발견하지 못한 사례 5개를 찾았습니다.
여기에는 시드 선택의 차이도 영향을 주었습니다. 두 비교 방법은 공식 테스트 581개 전체에서 시드를 고릅니다. 반면 SpecTrum은 목표 조건에 도달하는 입력을 선택해 총 59개를 사용했습니다. 이미 기존 테스트에서 거짓으로 평가된 조건은 목표에서 제외한다는 차이도 있습니다.

이 실험에서는 명세 정보를 활용한 생성 전략이 더 많은 불일치를 찾았습니다. 동시에, 선택한 시드와 목표 조건에 따라 놓치는 사례도 있다는 점을 확인했습니다.

한계 및 향후 계획

- 실제 네트워크에서의 발생 가능성은 미확인
  - 입력이 만들어질 수 있는지, 공격에 이용될 수 있는지
- 명세 준비와 목표 조건 선정에 수동 작업 필요
  - 예외 조건 추가 · 경로 조건 정리 · 목표 조건 선별
- 다음 단계: 준비 과정 자동화 → 더 큰 명세에 적용

합의 명세 버전	도입 및 개발 상태	상태 전이 관련 코드*
Capella	2023년 4월 도입	약 1,200줄
Gloas	2026년 Q4 도입 예정	약 2,300줄

현재 한계는 실제 네트워크에서의 발생 가능성을 확인하지 않았다는 점과, 테스트 생성을 준비하는 데 수동 작업이 많다는 점입니다.
첫째, 이번 실험에서는 주어진 상태와 블록을 클라이언트들이 다르게 처리한다는 사실을 확인했습니다. 이런 입력이 실제 네트워크의 실행 과정에서도 만들어질 수 있는지, 공격에 이용될 수 있는지까지는 확인하지 않았습니다.
둘째, 명세에 예외 조건을 추가하고 경로 조건과 실행 결과를 정리하는 작업은 사람이 수행했습니다. 거짓이 될 수 없는 조건을 제외하고 테스트 목표를 정하는 작업도 수동으로 진행했습니다.
이번 실험의 대상은 2023년 4월에 도입된 이더리움 합의 명세 버전인 Capella입니다. 이후 현재까지 세 차례의 후속 업그레이드가 적용됐습니다. 저희는 Capella의 범위에서는 명세와 테스트 생성을 수동으로 준비할 수 있었습니다.
다음 적용 대상인 Gloas는 현재 개발 중인 후속 합의 명세입니다. 공식 명세의 상태 전이 관련 코드를 이전 버전에서 이어받은 부분까지 합쳐 비교하면, Capella는 약 1,200줄, Gloas는 약 2,300줄로 약 1.8배입니다.
이처럼 정리하고 검토해야 할 코드가 늘어나므로, 예외 조건을 추출하고 경로 조건을 점검하며 테스트 목표를 선별하는 작업을 자동화하려 합니다. 이를 통해 더 큰 Gloas 명세에서도 저희 접근이 효과적으로 작동하는지 확인하는 것이 다음 목표입니다.

명세의 조건을 드러내면, 테스트의 빈틈을 겨냥할 수 있다

명세 풀어쓰기
↓
조건 커버리지 측정
↓
값의 유래를 활용한 테스트 생성

클라이언트 간 불일치 **27개** 발견

조건 **126개**를 추가로 거짓으로 평가

이 연구의 핵심은 명세를 풀어써 각 규칙의 조건을 드러내면, 기존 테스트가 다루지 않은 경우를 찾아 입력을 만들 수 있다는 것입니다.

저희는 명세를 풀어써 경로 조건과 실행 결과를 정리하고, 연산에 숨어 있던 예외 조건도 직접 적었습니다. 이 조건들의 커버리지와 값의 유래를 이용해, 기존 입력에서 어느 필드를 어떤 값으로 바꿀지 정했습니다.
그 결과, 기존 테스트에서 거짓으로 평가되지 않았던 조건 126개를 추가로 거짓으로 만들고, 다섯 클라이언트 사이에서 불일치 사례 27개를 발견했습니다.
이 연구에서는 실행 가능한 명세를 기대 결과를 계산하는 기준으로 사용하고, 동시에 다음에 검사할 입력을 만드는 정보로도 활용했습니다.

들어주셔서 감사합니다.

Old Versions